ХӨДӨЛМӨРИЙН ГАВЬЯАНЫ УЛААН ТУГИЙН ОДОНТ
МОНГОЛ УЛСЫН ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

F.CSM302 ҮЙЛДЛИЙН СИСТЕМ

БИЕ ДААЛТ 1 — ТАЙЛАН

2026-2027 оны хичээлийн жилийн намар

Сэдэв:	The Dining-Philosophers Problem
Хичээл заасан багш:	Г. Ганбат /F.CS10/
Хуваарилалтын код:	V-1-11
Гишүүд:	Б. Баттогтох /B241940031/
	/B221940703/
	/B231940154/
	/B241910011/
	/B241910034/

Улаанбаатар хот

2026 он

Гарчиг

Гарчиг 2
Хураангуй 4
1. Удиртгал 5
1.1 Зорилго 5
1.2 Ач холбогдол 5
1.3 Хамрах хүрээ 5
1.4 Ажлын хуваарилалт 5
2. Сурах бичгийн шаардлага 6
3. Онолын үндэс 7
3.1 Синхрончлолын асуудал ба critical section 7
3.2 Mutex ба condition variable 7
3.3 Monitor 7
3.4 Deadlock ба түүний дөрвөн нөхцөл 8
3.5 Naive шийдэл ба түүний deadlock 8
3.6 Monitor шийдэл (deadlock-аас сэргийлэх) 8
3.7 Зөв байдал ба starvation 8
4. Системийн зохиомж 9
4.1 Бүрэлдэхүүн 9
4.2 Philosopher-ийн төлөвийн диаграм 9
4.3 Өгөгдлийн бүтэц 9
4.4 Өгөгдлийн урсгал 10
4.5 UI-ийн зохиомж 10
5. Хэрэгжилт 11
5.1 Хэл, сан, орчин 11
5.2 Гол функцүүд 11
test(i) 11
pickup(i) 11
putdown(i) 11
5.3 Оролтын шалгалт 11
5.4 Зөв ажиллагааг хянах механизм 12
5.5 UI-ийн хэрэгжилт 12
5.6 Репозиторийн бүтэц 12
6. Туршилт ба UI дүрслэл 13
6.1 Юуг өөрчилж туршиж болох вэ 13
6.2 Юуг ажиглах вэ 13
6.3 UI-ийн дэлгэцийн зураг 13
7. Туршилтын арга 15

7.1 Орчин 15
7.2 Туршилтын төлөвлөгөө 15
7.3 UI-ийг гараар шалгах жагсаалт 15
8. Үр дүн 16
8.1 Үндсэн тестүүд (T1–T7) 16
8.2 Стресс тест (T8) 16
8.3 Алдаа ба захын оролт (E1–E8) 16
8.4 N-ээс хамаарал (m=5, t=20–60, e=20–60, seed=7) 17
8.5 Жишээ log (N=5, m=2, t=50–100, e=50–100, seed=42; эхний мөрүүд) 17
9. Хэлэлцүүлэг 18
9.1 Үр дүнгийн тайлбар 18
9.2 Хязгаарлалт 18
10. Дүгнэлт 18
Ашигласан материал 18
Хавсралт А. 20

Хураангуй

Асуудал. Dining-Philosophers бол олон процесс/thread хязгаартай нөөцийг (энд хэрэгсэл — fork) хуваалцахад үүсдэг синхрончлол ба deadlock-ийн сонгодог бодлого юм. Таван philosopher дугуй ширээний эргэн суугаад бодох (thinking) болон идэх (eating) үйлдлийг ээлжлэн хийх бөгөөд идэхийн тулд хоёр талын fork-ийг хоёуланг нь авах ёстой.

Шийдэл. Сурах бичгийн monitor шийдлийг C хэл дээр Pthreads ашиглан хэрэгжүүлэв. Нэг mutex (monitor-ийн түгжээ) болон philosopher бүрд нэг condition variable ашиглана. Philosopher зөвхөн хоёр хөрш нь идэхгүй байгаа үед л EATING төлөвт орно. Үүнээс гадна, philosopher-ийн тоо, thinking/eating хугацаа, seed, хурд, алгоритмыг тохируулах боломжтой HTML/JavaScript интерактив UI бүтээв. UI нь thinking, hungry, eating төлөв ба fork эзэмшилтийг бодит цагт харуулах бөгөөд Naive (зүүн→баруун) горимд deadlock-ийг илрүүлж харуулна.

Туршилт. Автомат 45 тест (7 үндсэн, 30 стресс, 8 алдаа/захын оролт) бүгд амжилттай. N=2…64 philosopher-ийн хувьд аль ч хоёр хөрш зэрэг идсэнгүй (violations=0), бүх philosopher тогтоосон тооны хоол идэж дууссан (deadlock-гүй).

Гол үр дүн. Нэгэн зэрэг идэж чадах philosopher-ийн дээд тоо ⌊N/2⌋-тэй нийцэв (N=12 үед 6). UI-ийн Naive горим deadlock-ийн механизмыг нүдэнд харагдахаар үзүүлнэ.

1. Удиртгал

1.1 Зорилго

Энэ ажлын зорилго нь сурах бичгийн Programming Project 11-ийн үндсэн шаардлагыг бүрэн хэрэгжүүлж, синхрончлолын механизм (mutex, condition variable, monitor) болон deadlock-аас сэргийлэх аргыг практикаар ойлгох, түүнийгээ өөр өгөгдөл, параметрээр давтан туршиж болох программ ба дүрслэлээр баталгаажуулах явдал юм.

1.2 Ач холбогдол

- Dining-Philosophers нь үйлдлийн системийн нөөцийн хуваарилалт, түгжээ (lock) -ийн зөрчлийн жижиг загвар. Үүнээс deadlock, starvation, зөв синхрончлолын ойлголтыг ойлгож болно.
- Бодит системд (өгөгдлийн сангийн row lock, файлын түгжээ, thread pool) ижил асуудал үүсдэг тул шийдэл нь шууд хэрэглэгддэг дадлага болно.
- Дүрслэл нь алгоритмыг "харж ойлгох" боломж олгож, зөв/буруу шийдлийг харьцуулах нотолгоо болдог.

1.3 Хамрах хүрээ

- Хамрах: Pthreads хэрэгжилт (Linux/macOS), CLI тохиргоо, автомат тест, HTML UI, тайлан ба гарын авлага.
- Хамрахгүй: өөр шийдлүүд (semaphore, resource hierarchy/Dijkstra, Chandy–Misra)-ийн бүрэн хэрэгжилт; Windows-ийн native Pthreads (WSL-ээр ажиллуулна); UI-ийн автомат тест.

1.4 Ажлын хуваарилалт

Гишүүн	Хариуцсан ажил	Commit / файл
Б. Баттогтох /B241940031/	C monitor хэрэгжилт, HTML UI, тест, тайлан, Manual	- src/philosophers.c - src/dining_philosophers.html - tests/, docs/
B221940703	-	-
B231940154	-	-
B241910011	-	-
B241910034	-	-

2. Сурах бичгийн шаардлага

Programming Project 11-ийн үндсэн шаардлага ба тэдгээрийг хэрхэн биелүүлснийг доорх хүснэгтэд харуулав.

№	Шаардлага	Хэрхэн биелүүлсэн	Хаана
1	Philosopher бүр тусдаа thread, thinking ба eating ээлжлэнэ	pthread_create-ээр N thread; thread бүр think → pickup → eat → putdown давтана	philosopher()
2	Pthreads mutex ба condition variable ашиглах	Нэг pthread_mutex_t lock ба pthread_cond_t wait_cv[i]	pickup, putdown
3	Monitor-ийн pickup/return үйлдэл, state[] (THINKING, HUNGRY, EATING)	state[] + test(); pickup/putdown нь lock дотор	test, pickup, putdown
4	Хөрш хоёр нэгэн зэрэг идэхгүй	test() хоёр хөршийг шалгана; инвариант шалгалт violations тоолуур	test, pickup
5	Thinking/eating хугацаа санамсаргүй	rand_r-ээр [min,max] мс; philosopher бүр өөр seed	random_between
6	Оролт, нөхцөл, параметрийг өөрчилж дахин туршина (БД1 нийтлэг шаардлага)	CLI: -n -m -t -e -s -q; UI: бүх параметр өөрчлөгдөнө	main, UI
7	Буруу, захын оролтыг шалгаж мэдээлнэ	Хязгаар шалгаад тодорхой мессеж, rc=2; UI-д улаан мессеж	main, UI
8	Багшийн UI: thinking/hungry/eating төлөв ба fork эзэмшил	HTML UI: өнгөт төлөв, fork байрлал, хүснэгт, log	UI
9	Өөр компьютерт Manual-аар дахин ажиллуулах	Manual 8.2, Makefile, README	docs/

3. Онолын үндэс

3.1 Синхончлолын асуудал ба critical section

Хэд хэдэн thread нэг хуваалцсан өгөгдөл, нөөцөд хандах үед үр дүн нь thread-үүдийн ээлжлэлээс хамаарах "race condition" үүсч болно. Үүнээс сэргийлэхийн тулд хуваалцсан өгөгдөлд хандах хэсгийг нэг дор зөвхөн нэг thread ажиллуулдаг болгоно.



Figure 7.5 The situation of the dining philosophers.

3.2 Mutex ба condition variable

- **Mutex**— түгжээ. lock хийсэн thread л critical section-д орно, unlock хийхээр бусад нь орж чадна.
- **Condition variable**— thread-ийг тодорхой нөхцөл биелэх хүртэл унтуулж, нөхцөл биелсэн үед сэрээх механизм. wait нь mutex-ийг атомаар суллаад унтдаг, сэрэхдээ дахин барьдаг.
- Нөхцөлийг үргэлж while давталтаар шалгана, учир нь thread сэрсний дараа нөхцөл өөрчлөгдсөн (spurious wakeup) байж болно. Манай pickup дээр while (state[i] != EATING) ийм байдлаар бичигдсэн.

3.3 Monitor

Monitor бол өгөгдөл ба түүнд хандах процедурыг нэг цогц болгож, нэг дор зөвхөн нэг thread монитор дотор ажиллахыг баталгаажуулдаг дээд түвшний синхончлолын бүтэц юм. C хэл monitor-ыг шууд дэмждэггүй тул mutex + condition variable-ийн хослолоор загварчилна. Манай монитор:

- state[N] — philosopher бүрийн төлөв
- wait_cv[N] — philosopher бүрд нэг condition variable (өөрийг нь л сэрээх боломж олгоно)
- lock — монитор доторх бүх үйлдлийг хамгаалах нэг mutex

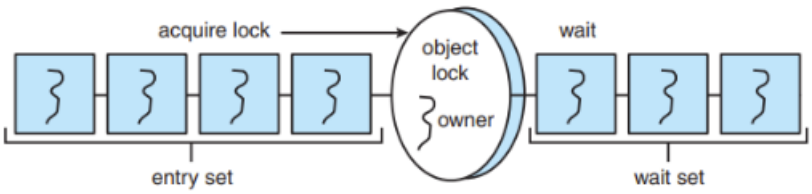


Figure 7.12 Entry and wait sets.

3.4 Deadlock ба түүний дөрвөн нөхцөл

Deadlock үүсэхэд дараах дөрвөн нөхцөл нэгэн зэрэг биелэх ёстой (Coffman):

Нөхцөл	Dining-Philosophers дахь утга
Mutual exclusion	Нэг fork-ийг нэг дор зөвхөн нэг philosopher барьна
Hold and wait	Зүүн fork-оо барьсан хэвээр баруун fork-ийг хүлээнэ
No preemption	Philosopher-ийн барьсан fork-ийг хүчээр булаах боломжгүй
Circular wait	P0→P1→…→P(N-1)→P0 гэсэн хүлээлтийн дугуй

3.5 Naive шийдэл ба түүний deadlock

Naive алгоритм: philosopher өлсөхөөрөө эхлээд зүүн fork-ийг авч, дараа нь баруун fork-ийг авна. Хэрэв N philosopher бүгд яг зэрэг өлсөж зүүн fork-оо авбал бүх fork эзэмшигдсэн болж, хэн ч баруун fork-ийг авч чадахгүй. Дөрвөн нөхцөл бүгд биелэгдэж deadlock болно. Манай UI-ийн Naive горим энэ байдлыг (бүгд HUNGRY + зүүн fork эзэмшсэн, ганц ч EATING байхгүй) илрүүлж DEADLOCK анхааруулга өгнө.

3.6 Monitor шийдэл (deadlock-аас сэргийлэх)

Monitor шийдэл нь "hold and wait" нөхцөлийг устгана: philosopher нэг fork-ийг тусад нь авч хүлээдэггүй, харин **хоёр fork-ийг нэг дор** авч чадах (хоёр хөрш нь идэж байхгүй) үед л EATING төлөвт орно. Хэрэв чадахгүй бол юу ч барихгүйгээр HUNGRY төлөвт унтана. Үүнээс fork-ийг дутуу барьж хүлээх нөхцөл үүсэхгүй тул circular wait боломжгүй.

3.7 Зөв байдал ба starvation

- **Аюулгүй байдал (safety):** test(i) зөвхөн хоёр хөрш EATING биш үед i-г EATING болгодог бөгөөд бүх өөрчлөлт нэг mutex дор хийгдэх тул хөрш хоёр зэрэг EATING байх боломжгүй. Програм үүнийг ажиллах үед violations тоолуураар шалгадаг.
- **Deadlock-гүй байдал:** дээрх 3.6-д тайлбарласнаар.
- **Starvation:** Энэ шийдэл starvation-ийг бүрэн үгүйсгэдэггүй. Philosopher i-ийн хоёр хөрш ээлжлэн идэж, i-ийг үргэлж хаасаар байх онолын боломж бий. Практикт санамсаргүй хугацаатай үед бага тохиолдох бөгөөд бид max_wait_ms хэмжилтээр хяналт тавьсан (8-р хэсэг).

4. Системийн зохиомж

4.1 Бүрэлдэхүүн

Компонент	Файл	Үүрэг
Pthreads хэрэгжилт (CLI)	src/philosophers.c	Бодит thread, mutex, condition variable. Тоон үр дүн ба инвариант шалгалт
Интерактив UI	src/dining_philosophers.html	Ижил state/test логикийг дискрет цагийн симуляцаар ажиллуулж дүрслэх; Naive горим
Build	Makefile	make, make test, make clean
Тест	tests/run_tests.sh	45 тест (T1–T8, E1–E8), tests/results.txt үүсгэнэ
Баримт	docs/, README.md	Тайлан, Manual

4.2 Philosopher-ийн төлөвийн диаграм

Одоогийн төлөв	Үйл явдал	Дараагийн төлөв	Fork
THINKING	Thinking хугацаа дуусав	HUNGRY	Fork барихгүй
HUNGRY	Хоёр хөрш EATING биш (test амжилттай)	EATING	Зүүн ба баруун fork-ийг хоёуланг эзэмшинэ
HUNGRY	Хөрш идэж байна	HUNGRY (хүлээнэ)	Юу ч барихгүй
EATING	Eating хугацаа дуусав	THINKING	Хоёр fork суллагдаж, хөршүүд test хийгдэнэ

Хөрш ба fork-ийн индекс: philosopher i-ийн зүүн fork нь i, баруун fork нь (i+1) mod N. Зүүн хөрш (i+N-1) mod N, баруун хөрш (i+1) mod N.

4.3 Өгөгдлийн бүтэц

Хувьсагч	Төрөл	Утга
state[i]	int	THINKING=0, HUNGRY=1, EATING=2
wait_cv[i]	pthread_cond_t	i-р philosopher хүлээх condition
lock	pthread_mutex_t	Монитор доторх бүх үйлдлийг хамгаална
meals_eaten[i]	int	i-р philosopher-ийн идсэн хоолны тоо
max_wait[i]	long	HUNGRY болсноос EATING хүртэл хамгийн их хугацаа (мс)
violations	int	Хөрш зэрэг идсэн тохиолдлын тоо (0 байх ёстой)
eating_now, max_eating	int	Нэгэн зэрэг идэж буй тоо, хамгийн их утга

4.4 Өгөгдлийн урсгал

1) Хэрэглэгч CLI параметр (эсвэл UI тохиргоо) оруулна → шалгалт.
2) N thread үүснэ, philosopher бүр thinking хугацаагаар унтна.
3) Philosopher HUNGRY болж pickup дуудна → lock авна → test.
4) Нөхцөл биелвэл EATING; үгүй бол pthread_cond_wait (lock-ийг суллаад унтна).
5) Eating дууссаны дараа putdown: THINKING болгож, зүүн ба баруун хөршид test хийж хүлээж байсныг сэрээнэ.
6) Бүх thread дуусмагц SUMMARY хэвлэж, гүйцэтгэл зөв бол rc=0, эсрэг тохиолдолд rc=1.

4.5 UI-ийн зохиомж

UI нь нэг HTML файл (гадаад сангүй).

Бүрэлдэхүүн:

1. SVG ширээ
2. тохиргооны самбар
3. төлвийн хүснэгт,
4. event log.

Симуляци нь 50 мс-ийн интервалтай tick() -ээр "симуляцийн цаг"-ийг нэмэгдүүлж, philosopher бүрийн таймерыг бууруулна. Хурдны сонголт (×0.5…×10) нь нэг tick-ийн симуляцийн хугацааг (50·хурд мс) өөрчилнө.

5. Хэрэгжилт

5.1 Хэл, сан, орчин

- C (gcc, -O2 -Wall -Wextra -pthread), POSIX Threads, rand_r, clock_gettime(CLOCK_MONOTONIC), nanosleep, getopt.
- UI: HTML5 + CSS + vanilla JavaScript + SVG; гадаад сан, сүлжээ шаардахгүй.
- Орчин: Ubuntu 24 (x86-64), GNU make.

5.2 Гол функцүүд

test(i)

Philosopher i өлсөж байгаа бөгөөд хоёр хөрш нь идэхгүй бол EATING болгож, i-ийн condition variable-ийг сэрээнэ. Үргэлж lock барьсан үед дуудагдана.

pickup(i)

Төлвөө HUNGRY болгож test хийнэ. Идэх эрх аваагүй бол while давталтаар унтана. Сэрсний дараа инвариантыг шалгаж, нэгэн зэрэг идэж буй тоог

putdown(i)

THINKING болгоод, зүүн ба баруун хөршид test хийж, тэднийг идэх боломжтой болсон бол сэрээнэ.

5.3 Оролтын шалгалт

Сонголт	Зөвшөөрөгдөх утга	Алдаа гарвал
-n	2…64 бүхэл тоо	Error: -n must be in 2..64, rc=2
-m	≥ 1	Error: -m must be >= 1, rc=2
-t min,max	0 < min ≤ max (мс)	Error: -t needs min,max with 0<min<=max (ms), rc=2
-e min,max	0 < min ≤ max (мс)	Error: -e needs …, rc=2
-s	бүхэл тоо	—
-q	—	—
Үл мэдэгдэх сонголт	—	Usage хэвлэж rc=2

N=1 үед нэг philosopher хоёр fork авах боломжгүй (зүүн ба баруун нь нэг fork) тул хүчингүй. Дээд хязгаар 64 нь статик массивын хэмжээ (MAX_N) ба thread-ийн тоог боломжийн түвшинд барих зорилготой.

5.4 Зөв ажиллагааг хянах механизм

- **violations** — pickup дотор сэрсний дараа хөрш EATING байвал нэмэгдэнэ. Зөв алгоритмд үргэлж 0.
- **max_concurrent_eating** — нэгэн зэрэг идэж байсан хамгийн их тоо; онолоор ⌊N/2⌋-ээс хэтрэх ёсгүй.
- **total = N × MEALS** — бүх philosopher хоол идэж дууссан эсэх (deadlock/унтаж үлдсэнийг илрүүлнэ).
- **max_wait_ms** — starvation-ийг шууд хэмжих үзүүлэлт.
- **Гарах код** — зөв бол 0, гүйцэтгэл буруу бол 1, оролтын алдаа 2. Энэ нь автомат тестийн үндэс.

5.5 UI-ийн хэрэгжилт

- **Төлвийн машин:** C хэрэгжилттэй ижил test/putdown логик, гэхдээ thread-ийн оронд дискрет цаг.**Seed:** mulberry32 PRNG — ижил seed ижил дараалал өгч, туршилтыг давтах боломжтой.
- **Naive горим:** HUNGRY philosopher зүүн fork чөлөөтэй бол авч, 500 мс "барьж" байгаад баруун fork-ийг авахыг оролдоно. Бүх N philosopher HUNGRY байж зүүн fork-оо барьсан, баруун fork-оо аваагүй бол DEADLOCK гэж зарлаж симуляцийг зогсооно.
- **Дүрслэл:** SVG. Philosopher-ийн өнгө = төлөв; fork чөлөөтэй бол ширээн дээр саарал, эзэмшсэн бол эзэмшигчийн хажууд эзэмшигчийн төлөвийн өнгөөр. Хүснэгт: төлөв, барьсан fork, идсэн тоо, хүлээсэн хугацаа.
- **Validation:** N 2…12, бүх хугацаа > 0, min ≤ max, seed тоо — алдаа гарвал улаан мессеж, симуляци эхлэхгүй.

5.6 Репозиторийн бүтэц

```
repo/
├── README.md
├── Makefile
├── src/
│   ├── philosophers.c
│   └── dining_philosophers.html
├── tests/
│   ├── run_tests.sh
│   └── results.txt
├── examples/
│   ├── sample_run_n5.log
│   └── scaling.txt
└── docs/ (Report, Manual)
```

6. Туршилт ба UI дүрслэл

6.1 Юуг өөрчилж туршиж болох вэ

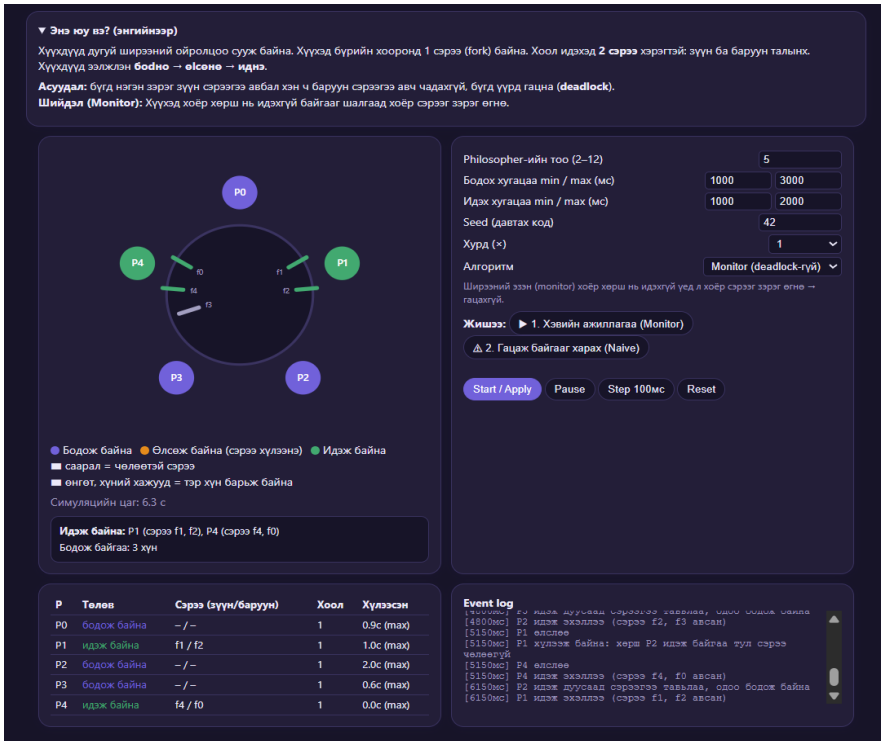
Параметр	Хязгаар	Хэрэглээ
Philosopher-ийн тоо	2–12 (UI), 2–64 (CLI)	Параллелизм, [N/2] хязгаарыг ажиглах
Thinking min/max	> 0 мс	Богино бол ширээ "өөрлөгдөөд" өрсөлдөөн ихэснэ
Eating min/max	> 0 мс	Урт бол хөршүүд удаан хүлээнэ
Seed	тоо	Туршилтыг яг давтах
Хурд	×0.5–×10	Хурдан/удаан ажиглах
Алгоритм	Monitor / Naive	Deadlock-гүй ба deadlock-той харьцуулах

6.2 Юуг ажиглах вэ

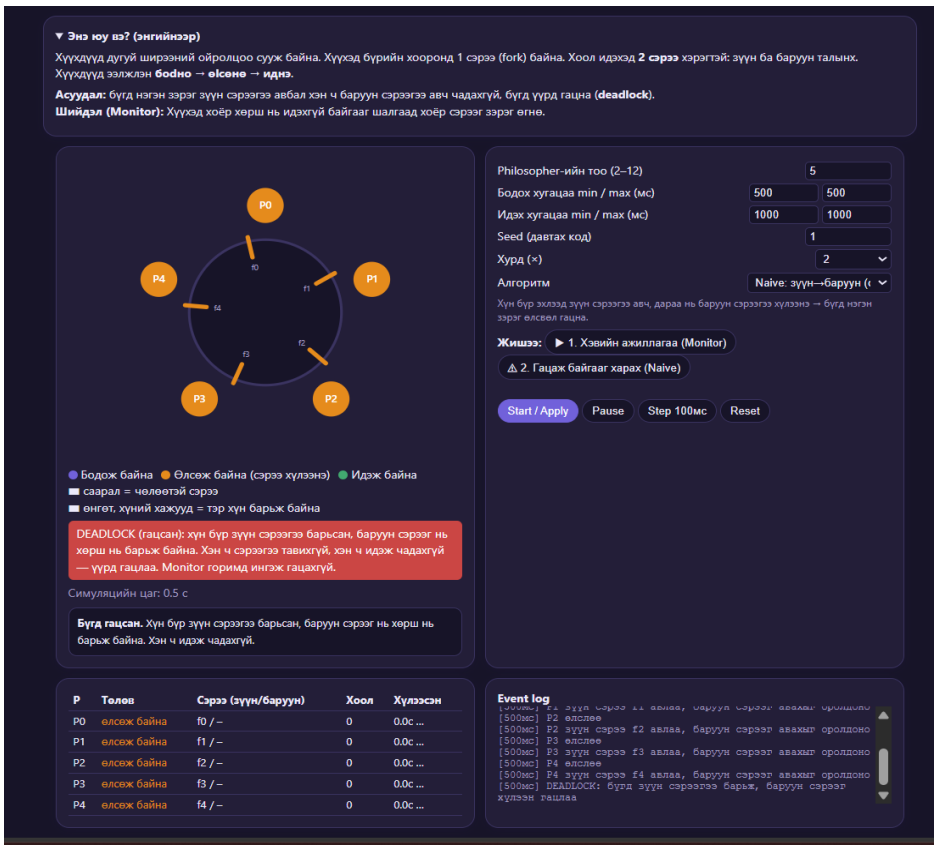
- Philosopher-ийн өнгө: цэнхэр THINKING → шар HUNGRY → ногоон EATING.
- Хөрш хоёр ногоон болохгүй (Monitor).
- Хоёр fork эзэмшсэн philosopher-ийн хажууд хоёр өнгөт fork харагдана; идэж дуусмагц fork ширээнд саарал болж буцна.
- Хүснэгтийн "Хоол" багана нэмэгдэж, "Хүлээсэн" багана hungry үед тоологдоно.
- Naive горимд бүгд шар болж зүүн fork-оо барьсан үед улаан DEADLOCK анхааруулга гарч цаг зогсоно.

6.3 UI-ийн дэлгэцийн зураг

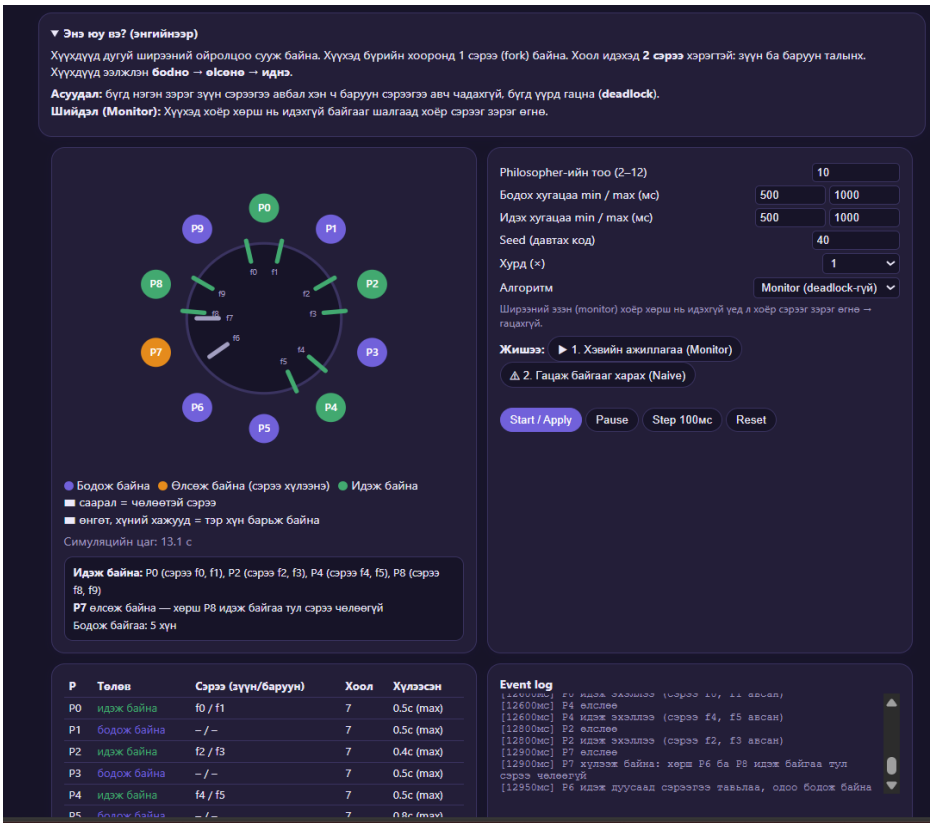
Дараах зураг нь бодит программаас (src/dining_philosophers.html) авсан дэлгэцийн зураг.



Monitor горим, N=5, thinking 1000–3000 мс, eating 1000–2000 мс, seed=42, симуляцийн цаг 6.3 с. P1 (сэрээ f1, f2) ба P4 (сэрээ f4, f0) идэж байна — хоёр нь хөрш биш; f3 чөлөөтэй (саарал). Хүснэгт, event log, "Одоо юу болж байна?" хэсэг ижил төлвийг харуулна.



Naive горим, N=5, thinking 500–500 мс, eating 1000–1000 мс, хурд ×2. 0.5 с-д бүх philosopher өлсөж зүүн сэрээгээ (f0…f4) барьсан, баруун сэрээгээ авч чадахгүй — улаан DEADLOCK анхааруулга гарч цаг зогссон.



Monitor горим, N=10, thinking 500–1000 мс, eating 500–1000 мс, seed=40, 13.1 с. P0, P2, P4, P8 зэрэг идэж байна; P7 өлсөж байна — хөрш P8 идэж байгаа тул сэрээ чөлөөгүй. Нэг дор 4 philosopher идсэн нь ⌊10/2⌋=5-аас хэтрээгүй.

7. Туршилтын арга

7.1 Орчин

Зүйл	Утга
OS	Ubuntu 24 (Linux x86-64)
Compiler	gcc, -O2 -Wall -Wextra -pthread (анхааруулгагүй хөрвүүлэгдсэн)
Build	GNU make
UI хөтөч	Firefox (UI гараар туршсан)
Хугацааны шинж	Бодит цаг (sleep), тиймээс хугацааны утга машин бүрд өөр

7.2 Туршилтын төлөвлөгөө

Бүлэг	Зорилго	Хүлээгдсэн үр дүн
T1–T7 үндсэн	Номын default (N=5), хамгийн бага/их N, сондгой N, тогтмол хугацаа, урт eating	rc=0, total=N×m, violations=0
T8 стресс	N=7, богино хугацаа, 30 өөр seed — race/deadlock-ийг барих	30 удаа бүгд rc=0
E1–E8 алдаа/захын	Буруу оролтыг татгалзах	rc=2 ба ойлгомжтой мессеж
UI гараар	Дүрслэл бодит логиктой нийцэх	7.3-ын хүснэгт

7.3 UI-ийг гараар шалгах жагсаалт

UI-д автомат тест хийгээгүй тул дараах жагсаалтаар гараар шалгана.

№	Алхам	Хүлээгдсэн	Дүн
U1	Default (N=5, Monitor) → Start	Хөрш хоёр хэзээ ч зэрэг ногоон болохгүй; хоол нэмэгдэнэ	PASS
U2	N=8 болгож Start/Apply	8 philosopher дүрслэгдэнэ, нэгэн зэрэг ≤4 ногоон	PASS
U3	Naive → Start	Бүгд шар болж DEADLOCK анхааруулга гарна, цаг зогсоно	PASS
U4	Pause → Step 100мс	Алхам алхмаар төлөв өөрчлөгдөнө	PASS
U5	Reset	Дэлгэц, хүснэгт, log цэвэрлэгдэнэ	PASS
U6	N=1 эсвэл N=20 оруулах	Улаан алдааны мессеж, симуляци эхлэхгүй	PASS
U7	Thinking min > max	Улаан алдааны мессеж	PASS
U8	Ижил seed-ээр хоёр удаа Apply	Эхний хэдэн event ижил дарааллаар гарна	PASS

8. Үр дүн

8.1 Үндсэн тестүүд (T1–T7)

Доорх утгууд нь tests/results.txt файлаас авсан бодит гаралт (make test).

Тест	Команд	Үр дүн	Дүн
T1 textbook default N=5	-q -n 5 -m 3 -t 20,60 -e 20,40 -s 42	total=15/15, violations=0, max_eat=2, max_wait=77мс	PASS
T2 N=2 (minimum)	-q -n 2 -m 5 -t 10,30 -e 10,30 -s 1	total=10/10, violations=0, max_eat=1, max_wait=19мс	PASS
T3 N=3 odd	-q -n 3 -m 5 -t 10,30 -e 10,30 -s 2	total=15/15, violations=0, max_eat=1, max_wait=38мс	PASS
T4 N=12	-q -n 12 -m 4 -t 10,40 -e 10,40 -s 3	total=48/48, violations=0, max_eat=6, max_wait=37мс	PASS
T5 N=64 (max)	-q -n 64 -m 2 -t 5,20 -e 5,20 -s 4	total=128/128, violations=0, max_eat=27, max_wait=34мс	PASS
T6 fixed times (min=max)	-q -n 5 -m 3 -t 10,10 -e 10,10 -s 5	total=15/15, violations=0, max_eat=2, max_wait=41мс	PASS
T7 long eating, short thinking	-q -n 5 -m 2 -t 1,2 -e 50,60 -s 6	total=10/10, violations=0, max_eat=2, max_wait=224мс	PASS

8.2 Стресс тест (T8)

N=7, m=5, thinking 1–10 мс, eating 1–10 мс, seed 100…129 — **30 удаа ажиллуулахад 30 нь PASS**. Нэг ч удаа deadlock, violations ≠ 0, total дутуу тохиолдол гараагүй. Хугацаа маш богино тул thread-үүдийн ээлжлэл ихээр хувирч race-ийг илрүүлэх боломж өндөр.

8.3 Алдаа ба захын оролт (E1–E8)

Тест	Команд	Хүлээгдсэн	Дүн
E1 N=1 rejected	-n 1	rc=2 + мессеж	PASS
E2 N=65 rejected	-n 65	rc=2 + мессеж	PASS
E3 N=abc rejected	-n abc	rc=2 + мессеж	PASS
E4 meals=0 rejected	-m 0	rc=2 + мессеж	PASS
E5 think min>max rejected	-t 300,100	rc=2 + мессеж	PASS
E6 eat negative rejected	-e -5,10	rc=2 + мессеж	PASS
E7 malformed range rejected	-t 100	rc=2 + мессеж	PASS
E8 unknown option rejected	-z	rc=2 + мессеж	PASS

8.4 N-ээс хамаарал (m=5, t=20–60, e=20–60, seed=7)

N	Нийт хоол	violations	Нэг дор идсэн max	⌊N/2⌋	Max хүлээлт (мс)	Нийт хугацаа (мс)
2	10/10	0	1	1	43	515
3	15/15	0	1	1	86	687
5	25/25	0	2	2	91	639
8	40/40	0	4	4	73	536
12	60/60	0	6	6	74	601

Нэг л ажиллалтын үр дүн тул хугацааны утгууд статистик дундаж биш. Харин "нэг дор идсэн max ≤ ⌊N/2⌋" ба "violations=0" нь бүх мөрд биелсэн.

8.5 Жишээ log (N=5, m=2, t=50–100, e=50–100, seed=42; эхний мөрүүд)

```
[    0ms] P0 thinking
[    0ms] P1 thinking
[    0ms] P2 thinking
[    0ms] P3 thinking
[    0ms] P4 thinking
[   56ms] P2 hungry
[   56ms] P2 eating
[   57ms] P1 hungry
[   73ms] P4 hungry
[   73ms] P4 eating
[   91ms] P0 hungry
[   99ms] P3 hungry
[  133ms] P4 thinking
[  133ms] P0 eating
[  151ms] P2 thinking
[  151ms] P3 eating
[  217ms] P0 thinking
[  217ms] P1 eating
[  219ms] P4 hungry
[  229ms] P2 hungry
[  242ms] P3 thinking
[  242ms] P4 eating
```

Log-оос ажиглах: P2 өлсөхөд (56мс) хөрш нь идэхгүй тул шууд идсэн; P1 өлсөхөд (57мс) хөрш P2 идэж байсан тул хүлээсэн, P2 дуусаад (151мс) P3 идсэн бол P1 нь P0 дуусахыг (217мс) хүлээж идсэн. Хөрш хоёр хэзээ ч зэрэг идээгүй.

9. Хэлэлцүүлэг

9.1 Үр дүнгийн тайлбар

- **Зөв байдал:** 45 тестийн бүх шалгуур (violations=0, total=N×m, rc) амжилттай. Ялангуяа стресс тест (богино хугацаа, олон өөр seed) race/deadlock-ийн нэг ч тохиолдол үзүүлээгүй нь mutex+condition variable-ийн хослол зөв ажиллаж буйг дэмжинэ (гэхдээ тест нь зөв байдлын албан ёсны баталгаа биш).
- **Параллелизм:** нэг дор идэх max нь N=2,3-т 1; N=5-д 2; N=8-д 4; N=12-т 6 — ⌊N/2⌋ дээд хязгаартай яг тохирно. Учир нь идэж буй philosopher бүр хоёр талын fork-оо эзэлдэг тул хоёр идэгчийн хооронд дор хаяж нэг philosopher байх ёстой.
- **Хүлээлт:** max_wait нь eating хугацааны хэмжээнд (≈20–160 мс) байсан. Хүлээлт нь хөршийн идэх хугацаанаас хамаардаг.
- **Naive ба Monitor:** UI дээр Naive горим deadlock-д автдаг, Monitor нь автдаггүй. Энэ нь онолын "hold and wait"-ийг устгах нөлөөг шууд харуулна.

9.2 Хязгаарлалт

- Monitor шийдэл starvation-ийг албан ёсоор үгүйсгэдэггүй. Хэрэв тодорхой philosopher-ийг хоёр хөрш нь ээлжлэн үргэлж хаавал удаан хүлээж болно. Бид ийм тохиолдол ажиглаагүй (max_wait бага) боловч онолын баталгаа биш.
- UI нь бодит Pthreads биш дискрет симуляци; цаг нь "симуляцийн цаг". Тиймээс C хэрэгжилтийн хугацаатай шууд тоон харьцуулалт хийхгүй.
- C програмд Naive горим байхгүй. Deadlock-ийг зөвхөн UI-д харуулна (бодит thread-ийг зориуд гацаах нь автомат тестэд тохиромжгүй).
- UI-ийн автомат тест байхгүй; гараар шалгах жагсаалтаар хязгаарлагдсан.
- Хугацааны хэмжилт нь нэг ажиллалт, нэг машин; статистик дундаж, итгэх завсар тооцоогүй.
- Windows-д native Pthreads байхгүй тул WSL эсвэл MinGW-w64 pthread хэрэгтэй.

10. Дүгнэлт

“The Dining Philosophers Problem”-ыг Pthreads-ээр бүрэн хэрэгжүүлж, mutex + condition variable ашигласан monitor шийдэл нь хөрш хоёр philosopher зэрэг идэхгүй, deadlock үүсэхгүй байхыг автомат тестээр (45 шалгалт) баталгаажууллаа. Параметрийг өөрчилж туршиж болох интерактив UI нь thinking/hungry/eating төлөв ба fork эзэмшлийг харуулж, Naive ба Monitor шийдлийн ялгааг, ялангуяа deadlock-ийг нүдэнд харагдахаар үзүүлнэ.

Ажлаас суралцсан зүйл: condition variable-ийг while-ээр шалгах, нэг үйлдлээр хоёр нөөцийг авах нь deadlock-аас сэргийлдэг, мөн зөв байдлыг инвариант тоолууруар автоматаар шалгадаг байх нь тестийг давтагдах боломжтой болгодог.

Ашигласан материал

1. Operating System Concepts, 10th edition (Silberschatz, Galvin, Gagne), Wiley, 2018 — Chapter 7: Synchronization Examples, The Dining-Philosophers Problem
2. GNU Make Manual: https://www.gnu.org/software/make/manual/
3. MDN Web Docs — HTML, CSS, JavaScript, SVG: https://developer.mozilla.org/
4. oscarsandford/dining-philosophers (GitHub) — бүтэц, санааг харьцуулахад судалсан, код хуулаагүй: https://github.com/oscarsandford/dining-philosophers
5. AI: Claude (Anthropic) — код, тест, UI, тайлангийн ноорог бэлтгэхэд ашигласан
6. Төслийн эх код ба README: https://github.com/battogtohbatushka-pixel/V-1-11-The-Dining-Philosophers-Problem

Хавсралт А.

Талбар	Мэдээлэл
Хуваарилалтын код	V-1-11
Төсөл	The Dining-Philosophers Problem
Repository	https://github.com/battogtohbatushka-pixel/V-1-11-The-Dining-Philosophers-Problem
Хэл/сан/орчин	C, Pthreads, gcc, GNU make; HTML/JS UI
Үндсэн дүрслэл	thinking/hungry/eating төлөв ба fork эзэмшил, philosopher болон хугацааны параметр

Гишүүн	Хариуцах ажил	Commit/file
Б. Баттогтох /B241940031/	C monitor хэрэгжилт (Pthreads: test, pickup, putdown)	src/philosophers.c
Б. Баттогтох /B241940031/	Build тохиргоо	Makefile
Б. Баттогтох /B241940031/	HTML UI: симуляци, Monitor/Naive, төлөв ба fork дүрслэл	src/dining_philosophers.html
Б. Баттогтох /B241940031/	Тест: үндсэн, стресс, алдаа/захын оролт	tests/run_tests.sh, tests/results.txt, examples/
Б. Баттогтох /B241940031/	Тайлан, Manual, README	docs/Report.docx, docs/Manual.docx, README.md